

Swig Wallet Assessment

Swig/ Anagram

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Aug 4, 2026 - Aug 31, 2026

SOURCE CODE
🔗 [swig-wallet](#)
📁 /swig-wallet-main-2/assertions/src/lib.rs
ASSESSED COMMIT ID
🔗 1735f54

SEVERITY BREAKDOWN



Summary

Swig team engaged Halborn to conduct a security assessment of their **swig** program from August 5th to August 31st , 2026. The security assessment was scoped to the Solana Programs provided in **swig_wallet** GitHub repository. Commit hashes and further details can be found in the Scope section of this report.

Swig is a role-based smart wallet protocol for Solana that lets multiple authorities operate a single on-chain wallet under fine-grained, permissioned rules. Each wallet is a program-derived account whose state encodes a set of *roles*, and each role binds an authenticated authority to a list of actions that scope what that authority may do — transferring SOL, managing SPL / Token-2022 tokens, staking, invoking whitelisted programs via CPI, managing other authorities, or operating sub-wallets.

The **Swig** team submitted a patch addressing a potential denial-of-service caused by Solana's Agave 4.2 upgrade, which lowers the account storage rent minimum. We reviewed the patch and added it to the audit additional scope; see the Scope section for details.

! Assessment Summary

Halborn was provided 19 days for the engagement and assigned 2 full-time security engineers to review the security of the Solana Program in scope. The engineers are blockchain and smart contract security experts with advanced smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the Solana Program.
- Ensure that smart contract functionality operates as intended.

In summary, **Halborn** identified some improvements to reduce the likelihood and impact of risks, most of which were addressed by the **Swig team**. The main ones were the following:

- Explicitly validate the `SwigWalletAddress` and reject self-mutating system program instructions.
- Enforce restricted-keys in sub-account signing.
- Correct the account layout in `TransferAssetsV1Instruction` and return an error in case of failure.
- Explicitly deny Swig re-entrant CPIs.
- Detect and exclude mutable fields from Token-2022 extensions from the hash verification by adjusting the accounting balance
- Prevent orphaned Sub-account state and asset PDAs with locked lamports on `CloseSwigV1`.
- Require the replacement-proof also for the `ProgramExecSession` authority type.
- Prevent the withdrawal v1 unreachable PDA denial-of-service.
- Make sure the `migrate_to_wallet_addres_v1` instruction is idempotent.
- Validate the destination when closing Swig.
- Gate the `set_rent_claimer_v1` on the migrated layout of Swig v2.
- Ensure the `transfer_assets_v1` accepts swig v2 layout only.
- Prevent permanent admin lockout of a Swig wallet.

Note: In commit `e772ef79ca4a9ed9b12e0aab25bfefc58402c7b7` the dependencies were upgraded removing the most of the previous existing entries in the cargo audit results. Further details can be found in the Automated Testing section of this report.

Scope

REPOSITORY



(a) Repository:  [swig-wallet](#)
(b) Assessed Commit ID:  1735f5498cbdfc6c78229b1057ac944cf074f96
Additional scope:
 f95c5abf43aa524a1eedaa1ba2ec97ca09a43251 ,  5b09d40c5fdf555e9d34ff2d67a28add13b87154

REMEDIATION COMMIT ID:



-  1b3c2bf1449259f68afa213bc616cb8969c1add7
-  5b09d40c5fdf555e9d34ff2d67a28add13b87154
-  906e502c42a16d1a3b987566d35a7c23118d720c
-  86e1f8fb4857b17fc62ca0bc3ee89f0922c29dea
-  ff2929bb780c14bc679ff03a20d4800b0b3b54a4
-  807d1d7d15adaba7a987c78add1638e0d720c25d
-  5e86ce94bbbe22fab418f06a92b184b65bb70c61
-  ffd9d8028a8bfa439ecb25acc29ddf4be438cafe
-  f3eaddb2c01c923c90d9e102765bfc37b31f754
-  2b139836cfee3e975e2ce850e6d8e37c4a9efba3
-  5091fbe2d493874ec02a8c7178ab26ffbd76414d
-  d210685246307adeb97c0e27433020fe77b2836d
-  N/A
-  35f2a550fcc03dceccb6bee2cd390b3fbe65d07e
-  1201494b4c2ef5024f6e0e364afd6d4c60378b3f
-  92b229f9512097fce8a4e38c10119684eedff671
-  afd68e36098682fe9b9c068d4bfaac8381976626
-  90836cb6fa4c2564ffbc2ecbbd244a2e2f445aa8
-  278c4722cf798241411558fbd6780f1b7cd25e1c
-  62539b8d0eb7d278f4edd576d1bb32ec57f629d4
-  87f1883d3cdfa38a7970b6060d5b8176e743bd41

Out-of-Scope: New features/implementations after the remediation commit IDs.

Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Sign_v2 omits owner and data length of SwigWalletAddress from its post-CPI integrity model	 Critical	10.0	SOLVED 08/19/2026
HAL-002	Missing restricted-keys enforcement in sub-account signing	 High	8.8	SOLVED 09/15/2026
HAL-003	Silent no-op in the SPL migration path of transfer assets due to desynchronized account layout	 Medium	5.3	SOLVED 08/31/2026
HAL-004	CPI reentrancy bypasses authority authentication in non-signing instructions via propagated is_signer bit	 Medium	5.0	SOLVED 08/31/2026
HAL-005	Token-2022 accounts with mutating account-level extensions may cause denial of service	 Medium	5.0	SOLVED 09/15/2026
HAL-006	CloseSwigV1 orphans V2 sub-account state and asset PDAs with permanently locked lamports	 Medium	4.7	SOLVED 08/19/2026
HAL-007	ProgramExecSession bypasses the replacement-proof check	 Medium	4.7	RISK ACCEPTED 09/10/2026
HAL-008	Withdrawal V1 unreachable PDA DoS via hardcoded role id search	 Low	4.0	SOLVED 08/20/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-009	Idempotency guard is unreachable in migration resulting in different issues	● Low	2.3	SOLVED 08/19/2026
HAL-010	Missing complete destination validation when closing swig	● Low	2.3	SOLVED 08/19/2026
HAL-011	Wallet-address guard bypass on V1 headers with permanent fund lockup in set rent claimer	● Low	2.3	SOLVED 09/04/2026
HAL-012	Broken migration guard in transfer assets allows draining rent-carrying v1 swig accounts to a non-canonical wallet pda	● Low	2.1	SOLVED 08/19/2026
HAL-013	Risk of permanent admin lockout of a Swig wallet	● Low	2.0	SOLVED 09/04/2026
HAL-014	Check_self_pda accepts non-canonical bump seeds, letting swig and sub-account pdas diverge from find_program_address	● Informational	1.6	ACKNOWLEDGED 08/24/2026
HAL-015	Swigwalletaddress account classification without pda ownership verification	● Informational	1.3	SOLVED 08/19/2026
HAL-016	Createv1 does not require a signature from the initial authority, allowing pda squatting or front-running to hijack wallet initialization	● Informational	1.3	SOLVED 08/20/2026
HAL-017	CreateSubAccountV2 allows SubAccountV2All squatting on pre-scoped future ids via counter	● Informational	1.1	SOLVED 09/04/2026
HAL-018	ManageAuthority Permits Granting Unbounded Administrative Permissions	● Informational	0.9	ACKNOWLEDGED 08/24/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-019	Migration omits program-ownership assertion on swig and system-program identity check	● Informational	0.8	SOLVED 08/19/2026
HAL-020	Transaction-scoped gatekeeper approval	● Informational	0.6	SOLVED 08/20/2026
HAL-021	Missing general action and authority identity deduplication	● Informational	0.6	SOLVED 09/08/2026
HAL-022	Compact instruction builder silently corrupts transaction semantics	● Informational	0.6	SOLVED 09/04/2026
HAL-023	Division by zero in recurring limit actions when window == 0	● Informational	0.5	SOLVED 08/19/2026
HAL-024	Wrong offset in actionloader::find_action causes action data to be deserialized from header bytes	● Informational	0.5	SOLVED 08/19/2026
HAL-025	V1 instruction builders emit undersized authority payloads, making sub-account and asset-transfer operations unusable for secp256k1 and secp256r1 authorities	● Informational	0.5	SOLVED 09/04/2026
HAL-026	Risk of rust undefined behavior	● Informational	0.4	SOLVED 09/10/2026

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's

integrity and addressing potential vulnerabilities introduced by code modifications.